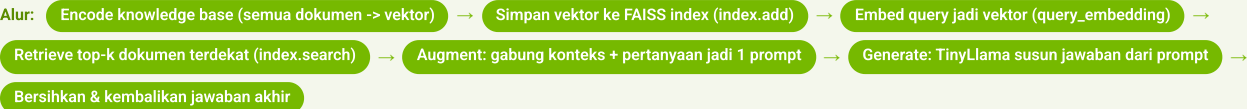


RAG Fundamentals

RAG (Retrieval-Augmented Generation) membekali LLM dengan konteks faktual dari knowledge base eksternal lewat embedding + similarity search sebelum menjawab, sehingga halusinasi berkurang dan jawaban bisa di-update tanpa melatih ulang model.



Kenapa Perlu RAG?

- LLM menjawab dari ingatan training saja
- Masalah 1: halusinasi (mengarang tapi meyakinkan)
- Masalah 2: knowledge cutoff (tak tahu kejadian terbaru)
- Solusi: bekali konteks faktual segar sebelum menjawab

↳ fakta terbaru / spesifik / domain

Pipeline RAG: Retrieve → Augment → Generate

- Retrieval: cari potongan info relevan dari knowledge base
- Augment: sisipkan konteks ke prompt
- Generation: LLM menyusun jawaban dari konteks
- Tidak mengubah otak LLM, hanya membekali konteks

```
# alur: Pertanyaan -> Retrieve -> Generate -> Jawaban
```

↳ inti seluruh sistem RAG

Embeddings & SentenceTransformer

- Embedding: teks -> vektor (deretan angka)
- Makna mirip -> vektor berdekatan
- Komputer bandingkan jarak antar-angka, bukan kata
- Model: all-MiniLM-L6-v2

```
embedder = SentenceTransformer('sentence-transformers/all-MiniLM-L6-v2')
v = embedder.encode(["capital of France"])
```

↳ ubah teks jadi angka yang bisa dicari

Similarity Search & FAISS

- FAISS = Facebook AI Similarity Search, pencari vektor super cepat
- IndexFlatL2: ukur jarak Euclidean (L2)
- Jarak makin kecil -> makna makin mirip
- Ambil top-k terdekat (notebook k=1; umumnya 3-5)

```
index = faiss.IndexFlatL2(embedding_size)
distances, indices =
index.search(query_embedding.astype('float32'),
k)
```

↳ cari dokumen relevan dari ribuan vektor

Knowledge Base

- Kumpulan dokumen/fakta sumber jawaban
- Tiap dokumen di-encode() jadi vektor
- Semua vektor masuk FAISS via index.add()
- Tambah dokumen = encode lalu add (tanpa latih ulang)

```
embeddings = embedder.encode(knowledge_base)
index.add(embeddings.astype('float32'))
```

↳ ingatan eksternal LLM

Generator LLM (TinyLlama)

- Generator = penulis jawaban: TinyLlama 1.1B Chat
- torch.float32 (presisi penuh) -> stabil di CPU
- Dibungkus pipeline("text-generation")
- temperature=0.3 (fokus), top_p=0.9 (nucleus sampling)

```
generator = pipeline("text-generation",
model=model, tokenizer=tokenizer,
max_length=512, do_sample=True,
temperature=0.3, top_p=0.9)
```

↳ menyusun jawaban akhir tanpa GPU

Langkah Retrieve

- Query di-embed jadi vektor (query_embedding)
- Dicocokkan ke FAISS (IndexFlatL2)
- Ambil top-k terdekat lalu ambil teks asli dari knowledge_base

```
query_embedding = embedder.encode([query])
dist, idx =
index.search(query_embedding.astype('float32'),
k)
docs = [knowledge_base[i] for i in idx[0]]
```

↳ tahap 1 pipeline

Langkah Augment

- Per kaya prompt dengan konteks hasil retrieve
- Format: bagian Context: lalu Question:
- Konteks + pertanyaan asli digabung jadi 1 prompt
- Notebook asli membungkusnya dalam chat template TinyLlama <|im_start|>

```
prompt = f"Context: {context}\nQuestion: {query}"
```

↳ jembatan retrieve -> generate

Tips & Batasan

- Chunk: terlalu besar -> kabur; terlalu kecil -> info hilang
- k: jumlah konteks diambil (notebook k=1, kadang butuh lebih)
- Kualitas embedding (all-MiniLM-L6-v2) menentukan relevansi
- RAG mengurangi halusinasi, BUKAN menghapusnya

↳ saat tuning kualitas jawaban

Quick-Ref Komponen RAG

Komponen	Library / Model	Tujuan
Embedding	SentenceTransformer (all-MiniLM-L6-v2)	Ubah teks jadi vektor
Index	FAISS (IndexFlatL2)	Cari vektor terdekat cepat
Generator	TinyLlama-1.1B-Chat (float32)	Susun jawaban akhir
Pipeline	Gabungan 1-3 (ask_question)	Retrieve -> augment -> generate
Lanjutan (Mod 06)	faiss-gpu + float16	RAG sama dipercepat GPU NVIDIA

Istilah Kunci

RAG — Retrieval-Augmented Generation: bekali LLM dengan konteks eksternal sebelum menjawab.

Embedding — Representasi teks sebagai vektor angka; makna mirip -> vektor berdekatan.

FAISS — Facebook AI Similarity Search; mesin pencari vektor terdekat yang cepat.

IndexFlatL2 — Index FAISS yang mengukur kemiripan via jarak Euclidean (L2).

top-k — Jumlah dokumen paling dekat yang diambil saat retrieve (notebook k=1).

Knowledge base — Kumpulan dokumen/fakta yang dibaca sistem RAG sebagai sumber jawaban.

Halusinasi — LLM mengarang jawaban yang meyakinkan tapi salah.

temperature — Pengatur keacakan jawaban; 0.3 = fokus/konsisten, makin tinggi makin acak.